



INDIVIDUAL ASSIGNMENT (1)

20%

SUBJECT NAME

Course: BIT1123 - Object Oriented Programming

Student Name: BALFAQIH ABOBAKR MOHAMMED ABOBAKR

Student ID: 202505010363

Class Code: 202605F0782

Programme: BCSSE

NRIC / Passport No : 14213550

Campus: Cyberjaya Campus

GitHub Repository

<https://github.com/abocom64-sketch/Balfaqih-Abobakr-OOP>

Contents

1. Introduction	3
2. Knowledge Gained from Each Tutorial	3
Week 1 - Java Fundamentals and Decisions	3
Week 2 - Classes, Objects, Constructors, and Methods	3
Weeks 3-4 - Inheritance and Method Overriding	3
Week 5 - Encapsulation.....	4
Week 6 - Reuse through Inheritance	4
Week 7 - Abstraction and Polymorphism	4
Weeks 8-9 - Collections, File I/O, and Exceptions.....	4
Week 10 - Swing GUI and Event-Driven Programming.....	4
3. Challenges Encountered	5
4. How I Overcame the Challenges	5
5. Improvements in My Java Programming Skills	6
6. Understanding of OOP Concepts	6
7. Future Learning Plans.....	7
8. Conclusion.....	8
9. GitHub Repository URL	8

1. Introduction

This report reflects on my progress throughout OOP tutorials. At the beginning, I understood a program mainly as a sequence of statements that produced an output. The tutorials gradually changed that view. I learned to represent a problem as a set of cooperating objects, each with its own state, responsibilities, and behaviour. This shift was important because it made larger programs easier to explain, extend, and test.

The GitHub repository records that development across Weeks 1-10. It contains small focused exercises rather than one disconnected final program. Looking back at these files allows me to see a clear learning path: fundamental syntax and decisions, classes and objects, inheritance, encapsulation, abstraction, polymorphism, collections, file input/output, exception handling, and a Swing graphical user interface.

My main goal was not only to make each program run, but to understand why object-oriented design is useful. I also practised professional habits by using consistent folders, meaningful names, documentation, version control, and a single repository for the course.

2. Knowledge Gained from Each Tutorial

Week 1 - Java Fundamentals and Decisions

I became familiar with the structure of a Java class, the main method, console output, variables, data types, and conditional statements. The grade example helped me understand that the order of an if-else chain matters. More specific or higher conditions need to appear before lower ones to avoid assigning the wrong grade.

Week 2 - Classes, Objects, Constructors, and Methods

I created a Student class with a name, age, and GPA, then initialized an object through a constructor. Methods such as displayInfo, study, and takeExam demonstrated that an object combines data with behaviour. I also learned the difference between defining a class and creating a particular instance of that class.

Weeks 3-4 - Inheritance and Method Overriding

The Person, Student, and Lecturer hierarchy taught me to move shared data and behaviour to a parent class. The subclasses reused the name and age fields while adding

student ID or subject information. Overriding introduce showed that the same operation can produce behaviour appropriate to the actual object type.

Week 5 - Encapsulation

I made the Student fields private and accessed them through constructors, getters, and setters. This exercise showed that an object's internal state should not be freely changed by unrelated code. Encapsulation creates a controlled boundary where validation rules can later be added without changing every class that uses the object.

Week 6 - Reuse through Inheritance

The Employee and Lecturer classes reinforced constructor chaining with super. Employee stored the common ID and name, while Lecturer added a subject and department. I learned that inheritance is valuable when there is a genuine is-a relationship, but it should not be used only to avoid writing a few repeated lines.

Week 7 - Abstraction and Polymorphism

The abstract Appliance class defined common power operations and required every appliance to implement operate. WashingMachine, Refrigerator, AirConditioner, and Microwave supplied their own versions. Referencing all of them through the Appliance type showed runtime polymorphism: the same method call selected different behaviour depending on the real object.

Weeks 8-9 - Collections, File I/O, and Exceptions

I used an ArrayList to store a dynamic sequence of tasks and a Scanner to collect input. BufferedWriter saved the tasks, and BufferedReader loaded them again. Try-with-resources demonstrated safe cleanup even when an error occurs. Catching IOException also taught me that a robust program must handle failure and communicate it clearly rather than stop unexpectedly.

Week 10 - Swing GUI and Event-Driven Programming

The Week 10 tutorial introduced a simple Swing quiz. The Questions class stores one question, two options, and the correct answer. QuizBattleGUI creates the window, displays the question, and provides two buttons for the user to choose an answer. An ActionListener checks the selected button and updates a result label to show whether the

answer is correct or incorrect. I learned that GUI code is event-driven because the program waits for the user to click a button before it responds.

The final Week 10 program also strengthened my understanding of separating concerns. Quiz data knows how to check an answer, while the window is responsible for presentation and interaction. This makes the model easier to test and the interface easier to change.

3. Challenges Encountered

One early challenge was deciding what should become a class, field, or method. It was easy to put all instructions inside main because that worked for small exercises. However, this approach became harder to follow as more data and behaviour were added. Inheritance created a second challenge: I needed to identify which members were truly shared and which belonged only to a subclass.

Encapsulation initially felt like extra work because private fields required accessor methods. The benefit became clearer when I considered invalid values and future changes. File handling introduced checked exceptions and resource management, while GUI programming added a different flow based on events and component state. In the quiz, my challenge was connecting each button to the ActionListener and making sure the correct feedback appeared after the user chose an answer.

Repository organization was another practical challenge. Files named Main.java occur in several tutorials and cannot be compiled together in one folder. Keeping each tutorial independent solved the naming conflict and made the learning sequence clearer.

4. How I Overcame the Challenges

I broke each problem into smaller responsibilities before coding. I first identified the nouns that could become objects, the data each object needed, and the actions it should perform. I then compiled and ran one tutorial at a time. Compiler messages helped me locate mismatched class names, missing methods, incorrect constructor arguments, and type errors.

For inheritance, I checked whether the subclass could reasonably be described as its parent. Shared members were placed in the superclass, while specialized fields stayed in subclasses. I used method overriding only when a subclass genuinely needed different behaviour. For file operations, try-with-resources provided a clear pattern for closing readers and writers, and exception messages gave the user useful feedback.

For the graphical quiz, I kept the question data in the Questions class and placed the Swing components and event handling in QuizBattleGUI. I tested both buttons to confirm that the result label displayed the correct feedback. Consistent folder names and a detailed README then made it easier to verify that every required tutorial and supporting file was present.

A useful lesson from this process is that debugging is not random trial and error. It is a cycle of reproducing a problem, reading evidence, isolating a smaller cause, applying one change, and testing again.

5. Improvements in My Java Programming Skills

My code has become more structured and readable. I now use constructors to create valid objects, methods to isolate behaviour, access modifiers to protect state, and collections instead of fixed variables when the amount of data can change. I am more comfortable reading compiler errors and selecting suitable standard-library classes rather than trying to implement everything from the beginning.

I also improved in naming and consistency. Descriptive class and method names explain intent, indentation reveals structure, and a small comment is used only when the reason is not obvious from the code. Compiling every tutorial independently gave me confidence that repository presentation and functional correctness support each other.

6. Understanding of OOP Concepts

Concept	My current understanding	Repository example
Encapsulation	Keep state behind a controlled public interface so an object can protect its own rules.	Week 5 Student

Concept	My current understanding	Repository example
Inheritance	Reuse and specialize a meaningful parent type through an is-a relationship.	Weeks 3-4 and 6
Abstraction	Expose essential operations while leaving implementation details to suitable concrete classes.	Week 7 Appliance
Polymorphism	Use one parent reference to call behaviour implemented differently by several object types.	Week 7 operate()

The most important connection between these ideas is responsibility. Encapsulation defines an object's boundary, abstraction defines what clients need to know, inheritance can organize related types, and polymorphism lets those types cooperate through a common contract. These concepts are strongest when they simplify a design, not when they are added only to satisfy terminology.

7. Future Learning Plans

My next goal is to move from small tutorial examples to tested applications with multiple packages and clearer architecture. I plan to learn Java interfaces in greater depth, generics, collection selection, custom exceptions, unit testing with JUnit, and build tools such as Maven or Gradle. I also want to practise refactoring so that I can improve an existing design without changing its behaviour.

For data-based applications, I plan to learn JDBC and database normalization, then connect Java objects to persistent records safely. For user interfaces, I want to compare Swing with JavaFX and apply layout, accessibility, and input-validation principles more systematically. Git branching, focused commit messages, and code review will be part of this practice because professional software development includes collaboration as well as programming.

A specific future project would be to extend the Week 10 quiz so questions can be loaded from a file or database, categories and difficulty can be selected, results can be stored, and model logic can be covered by automated tests. This would allow me to reuse the same OOP foundations in a more realistic system.

8. Conclusion

The tutorials gave me a connected foundation in Java and object-oriented thinking. I progressed from basic instructions and decisions to modelling data, protecting state, reusing behaviour, programming through abstract types, handling files and exceptions, and responding to GUI events. The most meaningful change is that I now think about responsibilities and relationships before writing code.

Challenges with program structure, inheritance, resources, and interface state were valuable because they taught me a repeatable problem-solving process. The completed repository represents both the technical outcomes and the improvement in how I organize, explain, and verify my work. I intend to build on this foundation through testing, databases, design patterns, and larger Java applications.

9. GitHub Repository URL

<https://github.com/abocom64-sketch/Balfaqih-Abobakr-OOP>